

# Trabalho – Programação III

Realizado por:

Tiago Pinto 54718

João Loios 55469

## Introdução

O nosso objetivo com este trabalho é implementar o jogo do desminador, em que se pretende identificar, num tabuleiro de  $N \times N$  casas, aquelas que têm uma mina. O programa que vai escrever será a parte "operacional" do jogo, e a interação será feita por texto. O programa deverá ler uma sequência de linhas (do standard input) que representam instruções e, para cada uma, dar a resposta adequada, linha-a-linha (no standard output).

## Implementação

Para realizar este trabalho optamos por usar OCaml, pois estávamos ambos mais á vontade com a linguagem.

Desenvolvemos este trabalho criando 2 tabuleiros, o campo\_real e o campo\_imaginario, o campo real contem as bombas, representadas a -1, e o resto das posições vazias, já o campo imaginário tem, no começo todas as posições vazias, e ao longo do jogo, de acordo com os inputs do utilizador, ele marca as posições com o valor de -1, que simboliza que está marcado com instrução mark, com o valor de -2, que simboliza que uma bomba explodiu (com a instrução step, e nesse caso o jogo irá terminar), ou com o valor de bombas em redor dessa casa,  $0 \leq N \leq 8$ , depois de step.

Note-se que optamos por representar a posições vazias com o valor 9, a fim de evitar conflitos com o valor 0, usado para identificar que tem 0 minas em redor.

## Organização do Código

O trabalho encontra-se organizado em várias funções que implementam os diferentes comandos do jogo, estas funções são usadas na função principal minesweeper, que consoante o input do

utilizador executa a função correspondente e usa os valores retornados por esta.

- **empty n:** Função que cria uma matriz de n linhas e n colunas e inicializa todos os valores dessa matriz a nove;
- **mine matrix r c:** Função que realiza o comando para colocar uma mina nas coordenadas (r, c). Esta coordenada ficará com o valor de -1, que significa que tem uma mina;
- **random matrix k:** Função que realiza o comando para colocar K minas em locais aleatórios;
- **count\_neighbors matrix r c:** Esta função conta o número de vizinhos que possuem mina para uma coordenada;
- **step matrixR matrixl r c:** Esta função representa a escolha de uma casa, onde se esta for igual a -1, significa que possui bomba, logo o jogador perdeu. Caso contrário, o jogo continua e irá ser chamado a função count\_neighbors para indicar o número de bombas próximas da coordenada escolhida;
- **mark matrix r c:** Função que realiza o comando para marcar uma coordenada, onde se julga haver uma bomba;
- **check\_done matrixR matrixl:** Função que verifica se todas as minas já foram encontradas, esta função conta as bombas que existem no campo real e guarda esse valor em count\_bombs, conta simultaneamente as bombas que estão corretamente marcadas e guarda esse valor em marked\_bombs, compara-os e caso sejam iguais manda um sinal à função principal para terminar o jogo, com sinal de vitória;
- **dump matrix:** Função usada para escrever o output da forma pedida do enunciado, usamos o campo\_imaginário para o fazer;
- **minesweeper:** Função principal que chama as outras funções, dependendo do comando introduzido pelo jogador no input. Foi usado o comando match, para fazer a associação do input com a função que é suposto ser chamada. Visto que o comando “empty”, tem de ser o primeiro a ser introduzido, esta função verifica se existe um tabuleiro criado, caso o utilizador execute um outro comando. Tal como é dito no enunciado, quando o comando é realizado com sucesso, o programa responde “ok”.

## Conclusão

Ao realizar este trabalho, conseguimos implementar todas as funcionalidades propostas no enunciado, como também, não encontramos um exemplo que originasse algum conflito com a lógica do programa. Por fim, gostamos de realizar este trabalho, e pretendemos melhorar os nossos conhecimentos e adquirir mais, futuramente.